

Exercise: Sentiment Analysis on IMDB Movie Reviews (Neural Networks + NLP Preprocessing)

1. Overview

In this exercise you will build a **neural network** that classifies IMDB movie reviews as **positive** or **negative**. Along the way you will practice a full, realistic NLP preprocessing pipeline — the kind of work that in real projects takes far longer than building the model itself.

You will work with the classic **IMDB Dataset of 50K Movie Reviews** on Kaggle:

- Dataset: <https://www.kaggle.com/datasets/lakshmi25npathi/imdb-dataset-of-50k-movie-reviews>
- Alternative source (same data, via Keras, if you cannot download from Kaggle): `tensorflow.keras.datasets.imdb` — but for this exercise you must use the **raw text CSV from Kaggle**, since the point is to practice preprocessing raw text yourself, not to use a pre-tokenized dataset.

The dataset contains 50,000 reviews (25,000 train / 25,000 test if you use the official split, or you can create your own split from the single CSV), evenly balanced between **positive** and **negative** labels.

2. Learning Objectives

By the end of this exercise you should be able to:

1. Load and explore a raw text dataset for a classification task.
2. Clean and preprocess raw text (HTML removal, punctuation, casing, stopwords, tokenization, stemming/lemmatization).
3. Convert text into a numeric representation suitable for a neural network (tokenizer + padded sequences, using an Embedding layer).
4. Build, train, and evaluate a **neural network** (not a classical ML model like Logistic Regression or Naive Bayes — those are for comparison only) for binary text classification.

5. Diagnose overfitting/underfitting using training curves.
6. Evaluate a classifier properly (accuracy, precision, recall, F1, confusion matrix) and interpret the results.
7. Run inference on new, unseen review text.

3. Requirements

- **You must use a neural network**, and for this exercise everyone builds the **same architecture**: an **Embedding layer + GlobalAveragePooling1D + Dense layers** (a "Neural Bag-of-Words" classifier). This is intentionally the *simplest* neural network architecture for text classification — no LSTM/GRU/CNN required. It is easy to reason about, trains in seconds/minutes on CPU, and is hard to get wrong, which is why it is the right starting point before you try anything fancier.
 - Library: **TensorFlow / Keras** (`tensorflow.keras`). Use exactly this, not PyTorch, for this exercise — the starter notebook and all steps below assume Keras.
 - Required layers, in order: `Embedding` → `GlobalAveragePooling1D` → `Dense(16, activation='relu')` → `Dropout(0.5)` → `Dense(1, activation='sigmoid')` .
 - A plain Logistic Regression / Naive Bayes / SVM on TF-IDF is **not** sufficient as your final model — you may build one only as an optional baseline for comparison in Step 10.
 - LSTM/GRU/Conv1D-based models are **not required** for this exercise. If you finish early and want a challenge, Step 10 (Bonus) gives you the option to try one, but it is not graded.
- You must implement your own **text preprocessing pipeline** (do not skip straight to a pretrained tokenizer without cleaning the text first).
- You must **split the data properly** into train / validation / test sets and avoid leaking test data into preprocessing steps that "learn" from data (e.g., fit your tokenizer/vectorizer only on the training set).
- You must report at least: accuracy, precision, recall, F1-score, and a confusion matrix on the test set.
- You must plot training vs. validation loss and accuracy curves.

4. Detailed Steps

Step 0 — Setup

- Download `IMDB Dataset.csv` from the Kaggle link above (or use the Kaggle API: `kaggle datasets download -d lakshmi25npathi/imdb-dataset-of-50k-movie-reviews`).
- Install/import the libraries listed in the starter notebook.

Step 1 — Load & Explore the Data

- Load the CSV into a pandas DataFrame.
- Inspect shape, column names, sample rows.
- Check class balance (`positive` vs `negative` counts) — plot it.
- Check for missing values and duplicate reviews; handle them.
- Look at review length distribution (word count) — plot a histogram. This will inform your padding length later.

Step 2 — Text Cleaning (Preprocessing Part A)

Implement a `clean_text()` function that:

- Removes HTML tags (reviews contain `<br />` tags — important, don't skip!).
- Removes URLs, if any.
- Lowercases all text.
- Removes punctuation and non-alphabetic characters (decide how to handle numbers, contractions like "isn't" — document your choice).
- Removes extra whitespace.

Apply it to the whole dataset and inspect a few before/after examples.

Step 3 — Tokenization & Linguistic Preprocessing (Preprocessing Part B)

- Tokenize the cleaned text into words (e.g. `nltk.word_tokenize`).
- Remove **stopwords** (e.g. using `nltk.corpus.stopwords`) — but think about whether removing negation words like "not", "no" is a good idea for sentiment analysis, and justify your decision in a markdown cell.
- Apply **stemming** (e.g. `PorterStemmer`) or **lemmatization** (e.g. `spaCy` or `WordNetLemmatizer`) — pick one, explain the trade-off in a markdown cell.

- Rejoin tokens back into a cleaned string (needed for the Keras tokenizer in the next step).

Step 4 — Encode Labels & Split the Data

- Map `positive` → 1, `negative` → 0.
- Split into train (70%), validation (15%), test (15%) — use `stratify` to preserve class balance.

Step 5 — Text Vectorization for the Neural Network

- Fit a `Tokenizer` (Keras) **only on the training set** — choose a sensible `num_words` (vocabulary size), e.g. 10,000–20,000.
- Convert train/val/test text to integer sequences with `texts_to_sequences`.
- Pad/truncate sequences to a fixed length with `pad_sequences` — choose the length based on your Step 1 histogram (e.g. covers ~90–95% of reviews without truncation).

Step 6 — Build the Neural Network

Build **exactly this architecture** using `tensorflow.keras.Sequential`:

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, GlobalAveragePooling1D,
Dense, Dropout

model = Sequential([
    Embedding(input_dim=VOCAB_SIZE, output_dim=EMBEDDING_DIM,
input_length=MAX_LEN),
    GlobalAveragePooling1D(),
    Dense(16, activation='relu'),
    Dropout(0.5),
    Dense(1, activation='sigmoid')
])

model.compile(loss='binary_crossentropy', optimizer='adam', metrics=
['accuracy'])
model.summary()
```

Why this architecture:

- `Embedding` turns each word-index into a learned dense vector.
- `GlobalAveragePooling1D` simply averages the word vectors of a review into one fixed-size vector — this *is* the neural network's only way of "reading" the sequence. It ignores word order, which is a real limitation, but it's simple, fast, and still beats classical bag-of-words models because the word vectors are learned for this task.
- `Dense(16, relu) → Dropout(0.5) → Dense(1, sigmoid)` is a small classifier head on top of the averaged review vector; `sigmoid` gives a probability of "positive" for binary classification.
- `binary_crossentropy` + `adam` is the standard loss/optimizer pair for binary classification with Keras.

Do not swap in an LSTM/GRU/Conv1D layer for the graded part of this exercise — keep the architecture above so everyone's results are comparable.

Step 7 — Train the Model

- Train with `model.fit()`, passing the validation set.
- Use callbacks: `EarlyStopping` (monitor `val_loss`) and optionally `ModelCheckpoint` to save the best model.
- Try a reasonable number of epochs (e.g. 10–20) and batch size (e.g. 32–64).

Step 8 — Evaluate

- Plot training vs. validation **loss** and **accuracy** curves (2 plots).
- Evaluate on the held-out **test set**: report accuracy, precision, recall, F1-score (`sklearn.metrics.classification_report`).
- Plot a confusion matrix (`sklearn.metrics.confusion_matrix` + `heatmap`).
- Discuss in a markdown cell: is the model overfitting? underfitting? What would you try next?

Step 9 — Inference on New Reviews

- Write a `predict_sentiment(text)` function that runs your **full pipeline** (clean → tokenize/stem/lemmatize → vectorize → pad → `model.predict`) on a brand-new review string typed by you.

- Test it on at least 3 reviews you write yourself (1 clearly positive, 1 clearly negative, 1 ambiguous/sarcastic) and comment on the results.

Step 10 (Optional / Bonus)

- Compare your neural network against a classical baseline (TF-IDF + Logistic Regression) — does the NN actually do better? By how much?
- Try pretrained word embeddings (GloVe) instead of a learned Embedding layer.
- Try a Bidirectional LSTM or stacking two LSTM layers and compare results.

5. Deliverables

Submit:

1. Your completed notebook (`imdb_sentiment_starter.ipynb` , renamed as you like), with all TODOs filled in and all cells run top-to-bottom without errors.
2. All markdown discussion cells filled in with your written answers/reasoning (not just code).
3. Final test-set metrics and the confusion matrix plot visible in the notebook output.